Carelink – Non-Profit Quiz/Tutor Bot

Institution Name: Medicaps University – Datagami Skill Based Course

Sr no	Student Name	Enrolment Number
1	Neha Tamboli	EN22CS301641
2	Riddhima Taose	EN22CS301642
3	Uday Patidar	EN23CS3L1023
4	Rohan Rawat	EN23CS3L1016
5	Naman Dwivedi	EN22CS301627

Group Name: 04D6

Project Number: DO-15

Industry Mentor Name:

University Mentor Name: Ajaj Khan

Academic Year: 2025-26

Table of Contents

1. **Introduction.**
   1.1. Scope of the document.
   1.2. Intended Audience.
   1.3. System overview.

2. **System Design.**
   2.1. Application Design.
   2.2. Process Flow.
   2.3. Information Flow.
   2.4. Components Design.
   2.5. Key Design Considerations.
   2.6. API Catalogue.
3. **Data Design.**
   3.1 Data Flow Description.
   3.2 Data Access Mechanism.
   3.3 Data Retention Policies.
   3.4 Data Migration.
4. **Interfaces.**
5. **State and Session Management.**
6. **Caching.**
7. **Functional Requirements.**
   7.1 State References User Authentication & Authorization.
   7.2 Application Configuration Management.
   7.3 Processing & Monitoring.
   7.4 System Operations.
   7.5 Docker.
   7.6 CI/CD Pipeline.
   7.7 CI/CD Strategy.
8. **Non-Functional Requirements.**
   8.1 Security Aspects.
   8.2 Performance Aspects.
9. **References.**
10. **GitHub.**

1. Introduction

1.1. Scope of the document

This document defines the high-level architecture and design for an AI-driven Non-Profit Quiz/Tutor Bot system. The system focuses on transforming donor communication data into an interactive learning experience using Artificial Intelligence. It leverages Large Language Models (LLMs) and vector databases to generate quiz questions, evaluate user responses, and provide contextual explanations.

1.2. Intended Audience

- Project Supervisors: To assess system design and feasibility.
- AI/ML Engineers: To understand LLM and vector database integration.
- Backend Developers: To implement FastAPI-based services.
- DevOps Engineers: To deploy using Docker and CI/CD pipelines.
- System Architects: To evaluate overall system architecture.

1.3. System overview

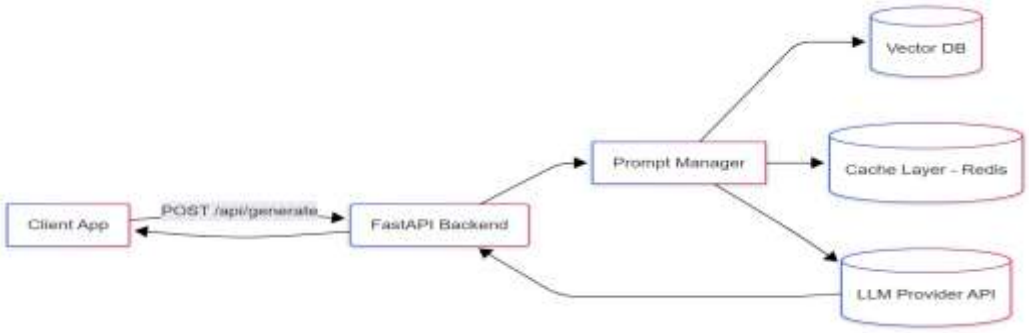
The AI Non-Profit Quiz/Tutor Bot is an intelligent system designed to convert donor emails into structured learning content. The system generates quiz questions based on donor communication, evaluates user responses using an LLM, and provides detailed explanations to improve understanding.

Additionally, the system includes a chatbot that allows users to interact with the knowledge base using natural language. The chatbot uses retrieval-augmented generation (RAG) to provide accurate and context-aware responses.

2. System Design

2.1. Application Design

The application follows a modular AI-driven architecture where different components handle specific functionalities such as quiz generation, answer evaluation, and chatbot interaction.



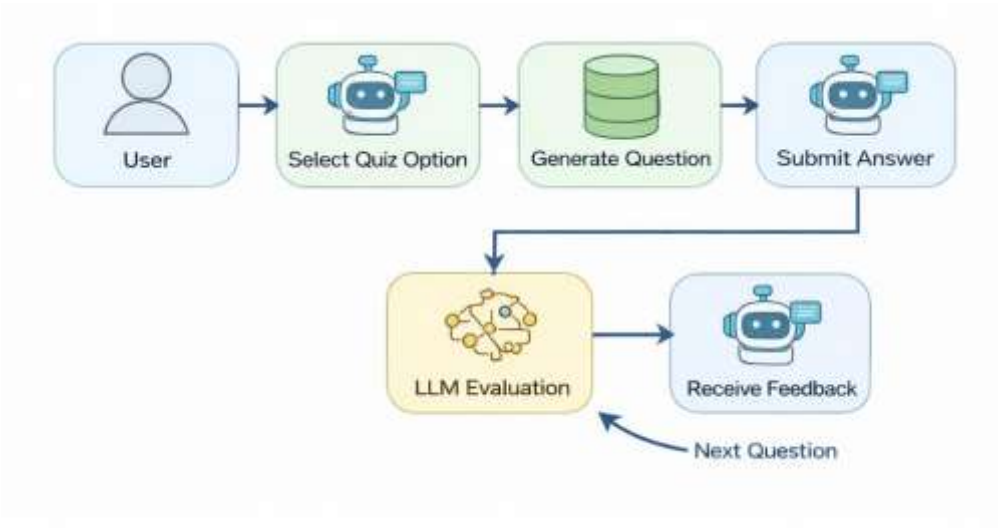
The system is divided into multiple layers including frontend, backend, AI processing modules, and data storage. The frontend provides a chatbot-like interface for quiz interaction. The backend, built using FastAPI, manages API requests and orchestrates communication between the quiz module, LLM evaluator, and vector database. The vector database stores donor emails as embeddings and retrieves relevant content based on user queries. The LLM is used for evaluating answers and generating explanations.

Components:

- Frontend Interface (Quiz + Chatbot)
- FastAPI Backend
- Quiz Generator Module
- LLM Evaluation Module
- Vector Database
- Chatbot (RAG-based)

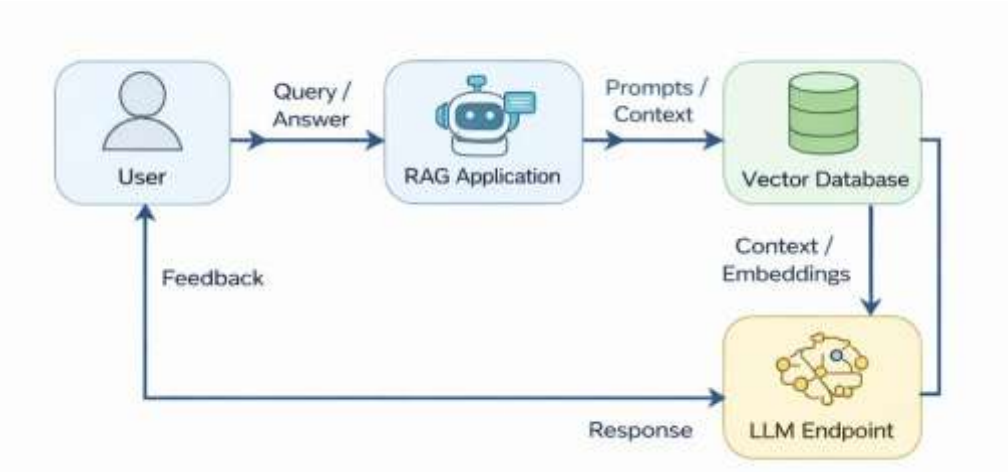
2.2 Process Flow

The process flow diagram represents the working of the AI-driven Non-Profit Quiz/Tutor Bot system, showing how a user interacts with the system in a continuous learning cycle. The process begins when the user accesses the system and selects a quiz option based on a specific domain such as education, healthcare, or clean water. Based on this selection, the system retrieves relevant donor-related information from the vector database and generates a meaningful quiz question. The user then reads the question and submits an answer through the interface. This response is sent to the Large Language Model (LLM), which evaluates the answer by analyzing its meaning, relevance, and completeness rather than relying on exact matching. The LLM then generates a detailed output that includes a score, a correct explanation, and feedback highlighting what the user missed. This feedback is displayed to the user to enhance understanding and improve knowledge of the topic. After this, the system uses the evaluation results to decide the next step in the quiz, either presenting a similar question if the user needs improvement or advancing to a more complex question if the performance is good. This creates a continuous loop of question generation, answer evaluation, and feedback, ensuring an adaptive and personalized learning experience for the user.



2.3 Information Flow

This diagram represents the working of a Retrieval-Augmented Generation (RAG) based AI system used in the Non-Profit Quiz/Tutor Bot. The process begins when the user sends a query or answer to the RAG application, which acts as the central controller. The application then converts the input into prompts and retrieves relevant context from the vector database, where donor-related data is stored as embeddings. This contextual information is passed to the LLM endpoint, which processes it along with the user input to generate an accurate and meaningful response. The response is then sent back to the user as feedback. This approach ensures that the system provides context-aware, precise, and informative outputs by combining stored knowledge with real-time AI reasoning.



Information Flow

2.4 Components Design

The system is composed of multiple modular components, each responsible for a specific function. This modular approach ensures scalability, maintainability, and efficient system performance

Components:

- **Frontend (Dashboard/UI):**

  Provides chatbot-style interface for quiz interaction and user input
- **FastAPI Backend:**

  Handles API requests, quiz logic, and system orchestration
- **Vector Database (ChromaDB/FAISS):**

  Stores donor emails and retrieves context using embeddings
- **Quiz Module:**

  Generates questions and manages quiz flow
- **LLM Module:**

  Evaluates answers and generates explanations

- **Chatbot Module:**

  Handles user queries using RAG-based approach

2.5 Key Design Considerations

The system is designed to ensure efficiency, scalability, and intelligent behavior. Special attention is given to AI accuracy and system performance.

Key Considerations:

- Modularity for easy maintenance
- Scalability using containerization
- AI accuracy through proper prompt engineering
- Fast retrieval using vector database
- User-friendly chatbot interface
- Secure API key management

2.6 API Catalogue

The system exposes APIs that enable communication between frontend and backend modules.

APIs:

- **LLM API:**
  Used for answer evaluation and chatbot response generation
- **Quiz API:**
  - `/question` → fetch quiz question
  - `/answer` → evaluate user answer
- **Chatbot API:**
  - `/chat` → handle user queries
- **Vector DB API:**
  Used internally for storing and retrieving embeddings

3. Data Design

3.1 Data Flow Description

The data flow begins with donor emails being stored in the vector database. When a user interacts with the system, relevant data is retrieved and used to generate quiz questions. User responses are processed by the LLM, and results are returned to the frontend.

Flow Points:

- Donor emails → Vector DB
- User interaction → Backend
- Backend → LLM evaluation
- Results → Frontend

3.2 Data Access Mechanism

- API-based secure access
- Environment variables for API keys
- Controlled access to vector database
- Backend validation of requests

3.3 Data Retention Policies

- Donor email data stored for learning purposes
- User responses may be stored for analytics
- Logs maintained for debugging
- Sensitive data handled securely

3.4 Data Migration

- Vector data can be migrated between environments
- Backend services can be redeployed using Docker
- Cloud migration supported (AWS, etc.)

4. Interfaces

- User Interface: Chatbot + Quiz Dashboard
- API Interface: FastAPI endpoints
- CLI: For backend and deployment

5. State and Session Management

Session management is handled at the backend level to track user progress during quiz interaction. The system maintains the current question, score, and user performance data to provide a personalized learning experience.

6. Caching

- Vector DB caching for fast retrieval
- API response caching for performance
- LLM response optimization

7. Functional Requirements

7.1 User Authentication & Authorization

- Optional login system
- Secure access control
- Session-based interaction

7.2 Application Configuration Management

- Environment-based configuration
- API key management
- Modular configuration

7.3 Processing & Monitoring

- Quiz generation
- Answer evaluation
- Chatbot responses
- System monitoring

7.4 System Operations

- Start/Stop backend
- Manage APIs
- Monitor system health

7.5 Docker

- Containerization of backend
- Easy deployment
- Environment consistency

7.6 CI/CD Pipeline

- Automated build and deployment
- GitHub integration
- Continuous updates

7.7 CI/CD Strategy

CI:

- Code validation
- Testing
- Build process

CD:

- Deployment automation
- Container execution
- System updates

8. Non-Functional Requirements

8.1 Security Aspects

- Secure API handling
- Environment variable usage
- Access control

8.2 Performance Aspects

- Fast response using vector DB
- Efficient LLM processing
- Scalable architecture

9. References

- Gen AI Quiz/Tutor Bot System
- LLM APIs (OpenAI)
- FastAPI Documentation
- Vector Database (ChromaDB/FAISS)

10. GitHub - https://github.com/NehaTamboli/carelink-project